

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения
высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет информационных технологий радиоэлектроники
Кафедра информационных систем

КУРСОВАЯ РАБОТА

по курсу Прикладная разработка на Java
на тему: Разработка системы подбора тренировок с учетом показателей
пользователей

Руководитель

(уч. степень, звание)

Метелкин А. С.

(фамилия, инициалы)

(подпись)

(дата)

Члены комиссии

Студент ИС-123

(группа)

Орлова П.С.

(фамилия, инициалы)

(подпись)

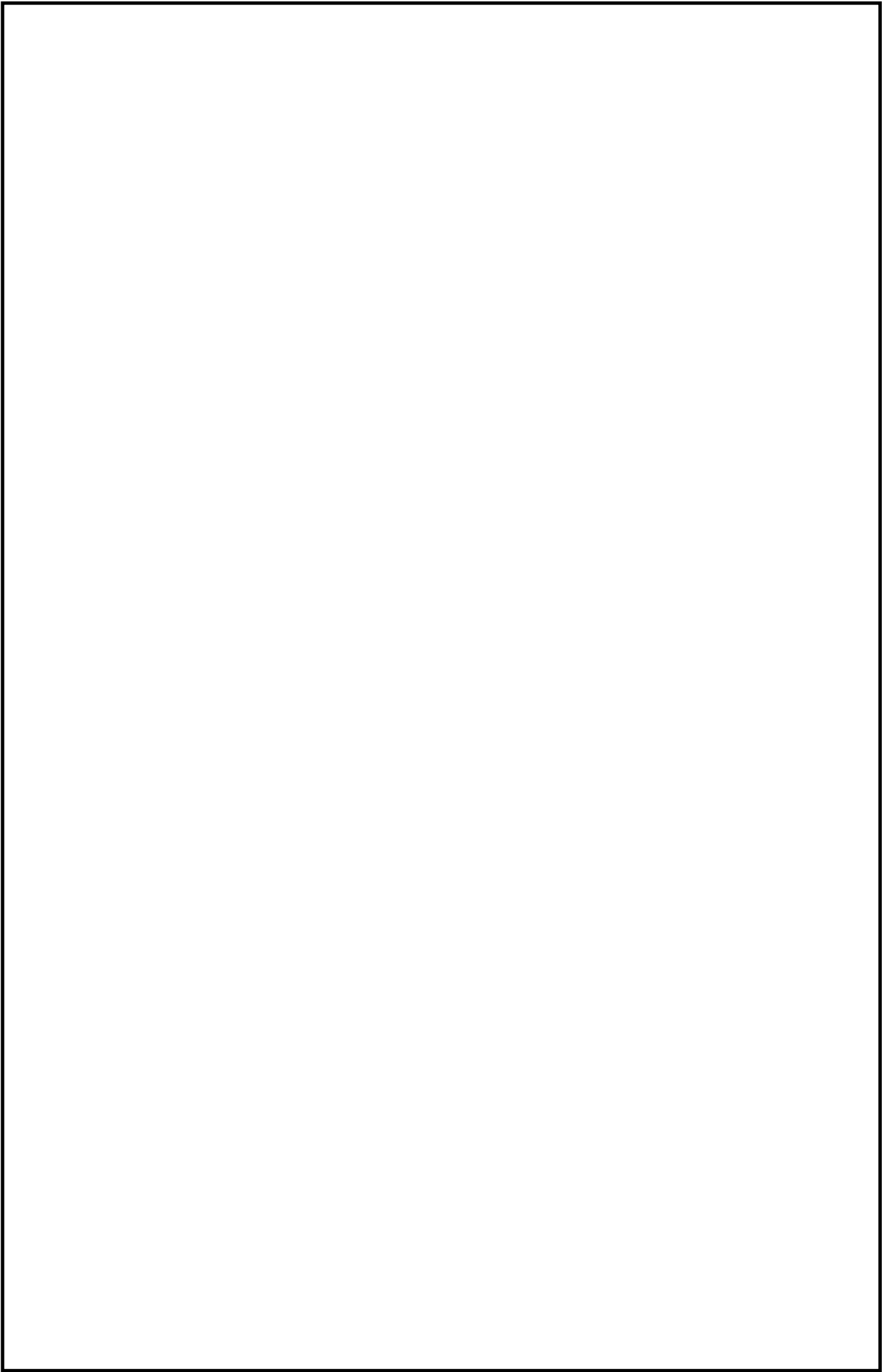
(Ф.И.О.)

(подпись)

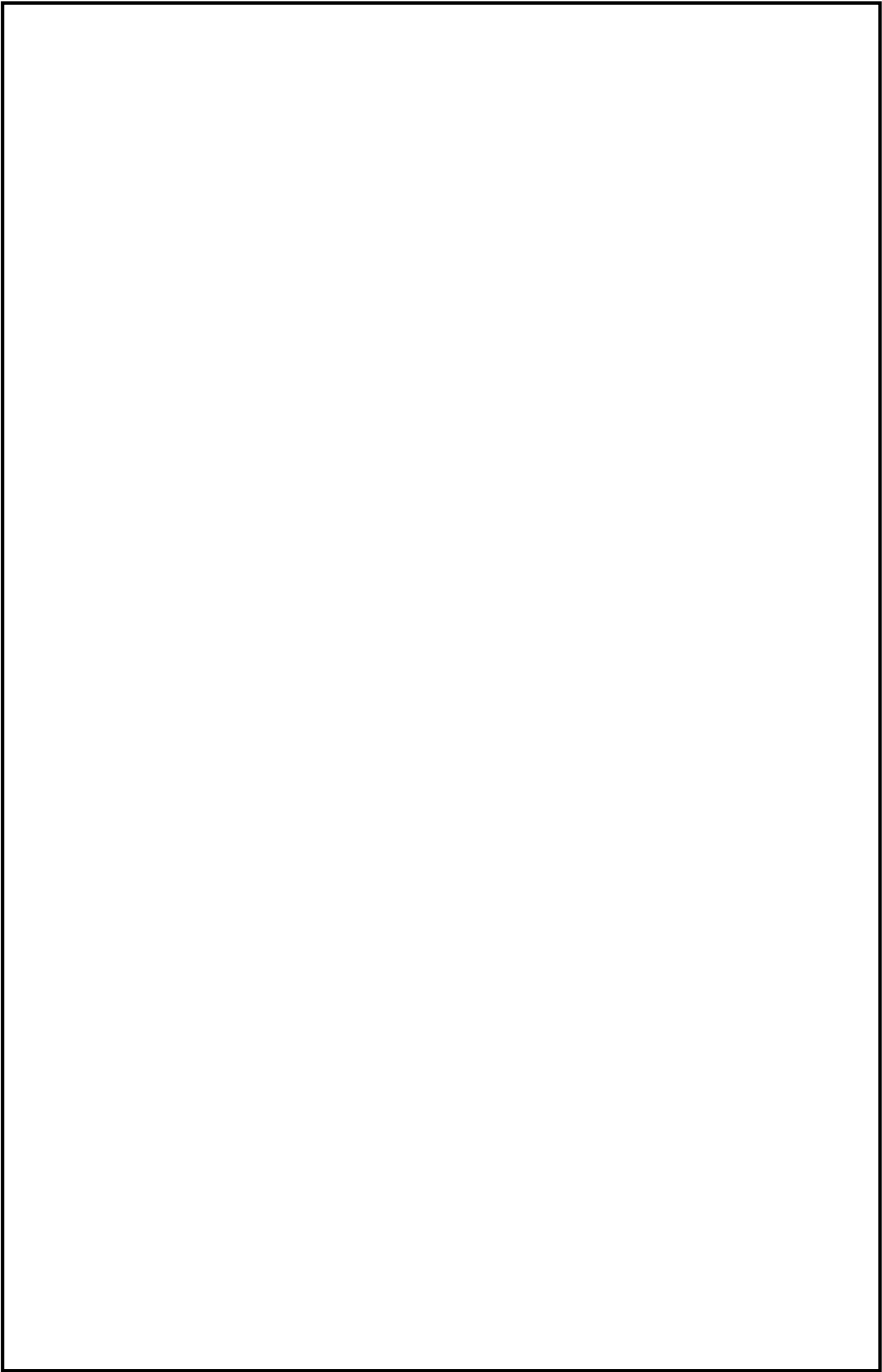
(Ф.И.О.)

(подпись)

(дата)



В курсовой работе отражена разработка информационной системы для автоматизированного подбора тренировочных программ с учетом индивидуальных показателей пользователей, реализовано хранение данных в реляционной базе данных, расчет индекса массы тела (ИМТ) и генерация детализированных отчетов в форматах (Excel, PDF, DOC). Спроектирована архитектура программы на основе принципов объектно-ориентированного программирования с применением паттернов проектирования. Создан интерфейс, позволяющий пользователю проходить регистрацию, отслеживать динамику веса, просматривать персонализированную программу тренировок и управлять составом упражнений. Реализованы алгоритмы подбора нагрузок: классический, весовой и интеллектуальный, а также различные способы визуализации: консольная, JavaFX и Swing. Проведено тестирование основных модулей. Разработанное приложение обеспечивает индивидуальный подход к формированию тренировочного процесса и удобный контроль спортивного прогресса.



Содержание

ВВЕДЕНИЕ.....	5
1. Анализ технического задания.....	7
1.1 Анализ существующих аналогов.....	7
1.2 Выбор технологий разработки.....	8
1.3 Использование паттернов проектирования.....	9
2. Проектирование программы.....	11
2.1 Проектирование системы на Java.....	12
2.2 Проектирование системы хранения данных.....	13
2.3 Проектирование графического интерфейса.....	14
3. Разработка программы.....	16
3.1 Разработка системы на Java.....	16
3.2 Разработка системы хранения данных.....	24
3.3 Разработка графического интерфейса.....	29
4. Тестирование.....	33
ЗАКЛЮЧЕНИЕ.....	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	38

					МИВУ.09.03.02 015 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Орлова П.С.			<u>Разработка</u> <u>системы подбора</u> <u>тренировок с учетом</u> <u>показателей</u> <u>пользователей</u>	Лит.	Лист	Листов
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								

					МИВУ.09.03.02 015 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Разработка системыподбора тренировок с учетом показателей пользователей	Лит.	Лист	Листов
Разраб.		Орлова П.С.						
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								

					МИВУ.09.03.02 015 ПЗ	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

ВВЕДЕНИЕ

В современных условиях укрепление здоровья и поддержание физической формы становятся приоритетными задачами для многих людей. Однако самостоятельный подбор тренировочных программ часто сопряжён с ошибками из-за отсутствия учёта индивидуальных показателей, уровня подготовки и конкретных целей. Универсальные планы не гарантируют эффективности и могут приводить к отсутствию прогресса или травмам. В связи с этим возрастает потребность в автоматизированных системах, способных анализировать данные пользователя и формировать персонализированные рекомендации, а также отслеживать динамику изменений в процессе тренировок.

Целью данного курсового проекта является разработка системы для подбора тренировочных программ с учётом индивидуальных показателей пользователей и реализации механизма контроля прогресса. Система должна обеспечивать регистрацию и авторизацию, расчёт индекса массы тела (ИМТ), генерацию программы на основе выбранной цели и локации, ведение журнала веса, а также экспорт отчётов в офисные форматы. Особое внимание уделяется построению модульной архитектуры с применением паттернов проектирования, обеспечивающих гибкость настройки алгоритмов подбора и поддержку различных вариантов пользовательского интерфейса.

Для достижения поставленной цели в рамках проекта необходимо решить следующие задачи:

- разработать архитектуру приложения с использованием интерфейсов для разделения логики программы;
- спроектировать и реализовать реляционную базу данных для хранения профилей пользователей, справочника упражнений и истории прогресса;
- реализовать алгоритмы расчёта ИМТ и подбора тренировок (классический, весовой и интеллектуальный);
- создать три варианта пользовательского интерфейса (консольный, Swing и JavaFX)

- внедрить функционал отслеживания динамики веса с автоматическим расчётом изменений;
- реализовать модуль экспорта данных в форматы Excel, PDF и Word.

Результатом работы является программное приложение, реализующее персонализированный подбор тренировок и контроль спортивных показателей. Разработанная система демонстрирует применение паттернов проектирования и принципов объектно-ориентированного программирования, обеспечивает удобное взаимодействие пользователя с данными и может использоваться как самостоятельный инструмент для планирования тренировочного процесса или в учебных целях.

					МИВУ.09.03.02 015 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

1 Анализ технического задания

Разработка приложения для подбора тренировочных программ и ведения дневника тренировок требует учёта особенностей предметной области, связанной с обработкой персональных данных пользователя, алгоритмическим подбором нагрузок и многоформатным экспортом отчётности. В спортивной и реабилитационной практике подобные системы используются для персонализации нагрузок, контроля прогресса и наглядного отображения динамики изменений. В связи с этим при проектировании приложения особое внимание уделяется разделению логики подбора программ, работы с базой данных и визуализации, а также обеспечению гибкости и расширяемости системы.

Предметная область включает обработку данных, параметров тренировок и истории прогресса. Исходные данные вводятся пользователем или загружаются из профиля, преобразуются во внутренние объекты модели и сохраняются в локальной СУБД. В процессе подбора программы происходит анализ целей, доступного инвентаря и физических параметров, результат чего должен корректно отображаться в выбранном интерфейсе без влияния на логику расчётов.

Анализ требований показал необходимость реализации модульной архитектуры с использованием интерфейсов, обеспечивающих слабую связанность между компонентами программы. Это позволяет разделить систему на независимые части: модели данных, слой доступа к базе (DAO), алгоритмы подбора, модули экспорта и варианты пользовательских интерфейсов. Такой подход упрощает сопровождение программы и позволяет расширять её функциональность без изменения существующего кода.

1.1 Анализ существующих аналогов

					МИВУ.09.03.02 015 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

В качестве существующих аналогов можно выделить такие приложения, как MyFitnessPal, Fitbod, Strong и Freeletics. Данные системы предназначены для планирования тренировок, учёта питания и отслеживания прогресса.

Большинство подобных решений реализуются как мобильные или веб-приложения с облачной синхронизацией, подписочной моделью и закрытой экосистемой. Основным преимуществом таких решений является удобство использования на смартфонах и наличие больших библиотек упражнений.

Однако подобные приложения зачастую ориентированы на коммерческое использование, не предоставляют доступа к внутренней логике подбора программ, не поддерживают работу в офлайн-режиме без аккаунта и не предусматривают академической демонстрации архитектурных решений. Кроме того, они редко предлагают несколько независимых вариантов интерфейса (консольный, классический десктопный, современный графический) в рамках одной кодовой базы.

В отличие от существующих аналогов, в рамках данной работы было принято решение реализовать настольное приложение на языке Java с акцентом на модульность, поддержку трёх типов интерфейсов, локальное хранение данных в СУБД Firebird и возможность экспорта отчётов в популярные офисные форматы без привязки к внешним сервисам.

1.2 Выбор технологий разработки

В качестве основной платформы разработки использовался язык программирования Java 21 [1][2]. Выбор Java обусловлен строгой типизацией, поддержкой современных возможностей ООП, кроссплатформенностью и развитой экосистемой библиотек для работы с базами данных, графикой и офисными форматами.

Для реализации пользовательских интерфейсов были использованы технологии Swing [3] и JavaFX [4]. Swing применялся для создания классического десктопного интерфейса с базовыми компонентами, а JavaFX —

					МИВУ.09.03.02 015 ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

для разработки современного графического интерфейса с поддержкой FXML-разметки, анимаций и реактивного обновления элементов. Дополнительно была реализована консольная версия, обеспечивающая работу в текстовом режиме без графической подсистемы.

Для хранения и обработки данных использовалась встраиваемая СУБД Firebird вместе с JDBC-драйвером Jaybird. Данный выбор обусловлен лёгкостью развёртывания, надёжностью транзакционной модели и отсутствием необходимости в установке отдельного серверного ПО.

Для работы с внешними форматами и сериализацией данных применялись библиотеки Apache POI (генерация Excel-отчётов), iText (создание PDF-документов) и Gson (работа с JSON). Для сборки проекта и управления зависимостями использовалась система Apache Maven, что позволило автоматизировать компиляцию, упаковку в JAR и управление сторонними библиотеками.

Использование данного стека технологий обеспечило стабильность работы приложения, возможность работы в автономном режиме и соответствие требованиям технического задания по поддержке нескольких интерфейсов и форматов экспорта.

1.3 Использование паттернов проектирования

Важным требованием технического задания являлось применение архитектурных паттернов проектирования. Их внедрение позволило снизить связанность модулей, упростить тестирование отдельных компонентов и обеспечить возможность масштабирования системы.

В рамках проекта используется паттерн Strategy. Он применяется для динамического выбора алгоритма подбора тренировочной программы в зависимости от цели пользователя (набор массы, похудение, поддержание формы) и условий тренировок (зал, дом). Все стратегии реализуют общий

интерфейс, что позволяет контексту приложения переключать алгоритмы во время выполнения без изменения основной бизнес-логики.

Для создания компонентов пользовательского интерфейса, объектов доступа к данным (DAO) и модулей экспорта используется паттерн Factory. Фабрики централизуют процесс инициализации объектов, избавляя код от множественных условных конструкций и упрощая добавление новых типов интерфейсов или форматов выгрузки отчётов.

Для организации взаимодействия между алгоритмами сортировки и визуализацией используется паттерн Observer. Во время сортировки алгоритм передаёт текущее состояние массива в модуль визуализации, который отображает изменения пользователю. Благодаря этому сортировка не зависит от конкретного способа отображения данных.

Для организации реактивного обновления состояния приложения применяется паттерн Observer. При изменении ключевых параметров пользователя (например, записи нового веса или обновления цели) система автоматически уведомляет все зависимые компоненты: панели статистики, графики прогресса и журналы тренировок. Это обеспечивает согласованность данных в разных частях интерфейса без жёсткой привязки модулей друг к другу.

Особое внимание уделено требованию расширяемости. Архитектура спроектирована так, чтобы добавление новой цели тренировок, формата экспорта или варианта интерфейса требовало только создания нового класса, реализующего соответствующий интерфейс, и его регистрации в фабрике. Существующий код при этом остаётся неизменным, что соответствует принципу открытости/закрытости.

В результате анализа технического задания были определены основные подсистемы приложения, механизмы их взаимодействия и правила расширения. Выбранный подход, основанный на интерфейсах и паттернах проектирования, полностью соответствует поставленным требованиям и обеспечивает надёжность, гибкость и удобство сопровождения программного продукта.

					МИВУ.09.03.02 015 ПЗ	Лист
						10
Изм.	Лист	№ докум.	Подпись	Дата		

2 Проектирование программы

После проведения анализа технического задания была определена общая архитектура разрабатываемой системы. Основной задачей проекта являлось создание приложения для подбора тренировочных программ и ведения дневника тренировок, поддерживающего динамическую адаптацию нагрузок, работу с локальной СУБД и экспорт отчётности в различных форматах. Дополнительно необходимо было обеспечить независимость алгоритмов подбора от интерфейса, возможность работы в трёх режимах отображения и дальнейшее расширение программы без изменения существующей логики.

На начальном этапе разработки была сформирована последовательность проектирования основных компонентов системы. В процессе разработки выполнялось создание структуры проекта, проектирование интерфейсов сущностей и DAO, реализация алгоритмов подбора тренировок, разработка системы визуализации и экспорта, настройка подключения к СУБД Firebird, а также интеграция паттернов проектирования.

На рисунке 1 представлена схема взаимодействия интерфейсов.

					МИВУ.09.03.02 015 ПЗ	Лист
						11
Изм.	Лист	№ докум.	Подпись	Дата		

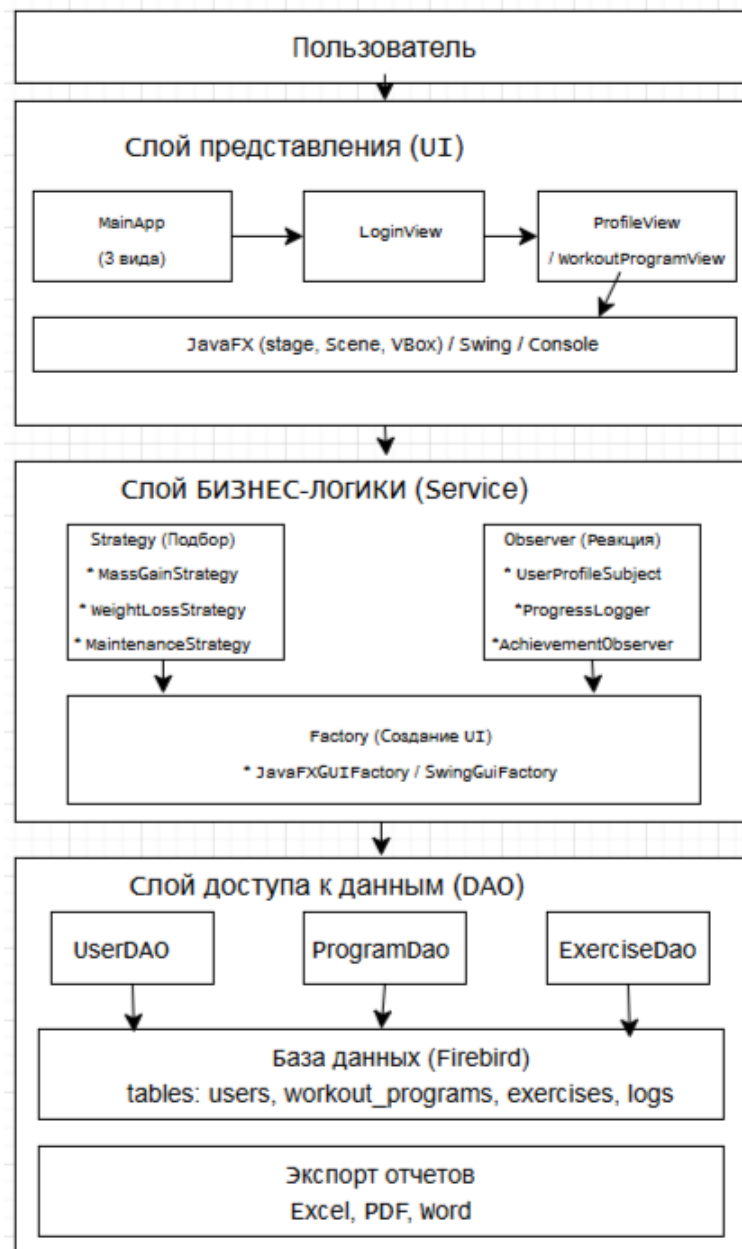


Рисунок 1 – Схема взаимодействия интерфейсов

2.1 Проектирование системы на Java

При проектировании системы было принято решение использовать объектно-ориентированный подход и многоуровневую архитектуру. Основной целью являлось разделение программы на независимые подсистемы, каждая из которых отвечает за строго определённую задачу.

В процессе проектирования были выделены следующие основные подсистемы:

- подсистема управления данными пользователей и тренировок (Model/DAO);
- подсистема алгоритмического подбора программ (Strategy);
- подсистема хранения и экспорта данных (Database/Export);
- подсистема пользовательских интерфейсов (View);
- подсистема управления созданием объектов (Factory).

Для обеспечения взаимодействия между компонентами были спроектированы интерфейсы, определяющие общий механизм работы модулей. Такой подход позволил сделать архитектуру гибкой и обеспечить возможность дальнейшего расширения программы.

Подсистема алгоритмов подбора проектировалась как независимый модуль, работающий только с параметрами пользователя (рост, вес, цель, локация). При этом алгоритмы не зависят от графического интерфейса или способа хранения данных.

Для организации реактивного обновления состояния приложения был предусмотрен механизм паттерна Observer. При изменении ключевых показателей (например, записи нового веса) система автоматически уведомляет зависимые компоненты: панели статистики и журналы прогресса.

Также на этапе проектирования была предусмотрена система фабрик объектов. Использование фабрик позволило централизовать создание компонентов программы и исключить зависимость основного модуля от конкретных реализаций интерфейсов, DAO-классов и модулей экспорта.

2.2 Проектирование системы хранения данных

Одним из требований технического задания являлась поддержка локальной базы данных и возможности экспорта отчётов. В связи с этим при проектировании была предусмотрена отдельная подсистема хранения данных.

Основной задачей данной подсистемы являлось обеспечение универсального механизма работы с информацией независимо от источника или

формата вывода. Для этого было принято решение использовать интерфейсы доступа к данным (DAO) и отдельный модуль инициализации СУБД.

На этапе проектирования были определены основные форматы взаимодействия с данными:

- хранение профиля и истории тренировок в СУБД Firebird;
- экспорт отчётов в форматы Excel (.xlsx), PDF (.pdf) и Word (.doc);
- внешняя конфигурация подключений через файл properties.

Внутри программы данные должны были храниться в виде объектов моделей (User, WorkoutProgram, UserProgress). Поэтому независимо от способа сохранения информация после загрузки из БД преобразуется во внутреннее представление в оперативной памяти.

При проектировании системы хранения также учитывалась возможность дальнейшего расширения. Архитектура должна позволять добавление новых форматов экспорта или смену СУБД без изменения существующей бизнес-логики. Для этого каждый модуль экспорта проектировался как отдельный независимый класс, взаимодействующий с системой через общий интерфейс генерации отчётов.

2.3 Проектирование графического интерфейса

При проектировании пользовательского интерфейса основной задачей являлось создание удобной системы управления тренировочным процессом и обеспечение наглядного отображения прогресса.

На этапе проектирования было принято решение реализовать несколько вариантов визуализации:

- консольную визуализацию;
- графический интерфейс на основе Swing;
- графический интерфейс на основе JavaFX.

Использование нескольких вариантов позволило обеспечить универсальность программы и продемонстрировать различные подходы к

построению пользовательского взаимодействия.

Консольный интерфейс проектировался как наиболее быстрый способ работы с программой. Основной задачей данного варианта являлся вывод текстовых меню для регистрации, выбора цели и просмотра подобранной программы. Дополнительно предусматривалась валидация вводимых данных и обработка ошибок подключения к БД в текстовом режиме.

Интерфейс Swing проектировался как стандартная оконная система. В рамках данного варианта предусматривалось использование JFrame, JPanel и стандартных компонентов для отображения профиля, таблиц с упражнениями и кнопок управления. Swing-версия должна была обеспечить кроссплатформенность и минимальные требования к ресурсам.

Основным графическим подсистемой проекта была выбрана JavaFX, поскольку данная технология предоставляет современные средства компоновки, поддержку CSS-стилизации и реактивное обновление элементов. При проектировании интерфейса JavaFX были определены основные экраны:

- экран авторизации и регистрации;
- главный экран с параметрами профиля и целью;
- экран подобранной программы тренировок;
- экран журнала прогресса и экспорта отчётов.

Визуализация данных проектировалась в виде таблиц и карточек упражнений, где каждый элемент содержит описание, количество подходов и рекомендации. Дополнительно была предусмотрена обработка событий изменения веса для автоматического пересчёта ИМТ и обновления рекомендаций.

Особое внимание при проектировании интерфейса уделялось независимости отображения от логики подбора. Графический интерфейс должен был только отображать состояние программы и принимать пользовательские команды, не вмешиваясь в алгоритм расчёта нагрузок. Благодаря этому обеспечивается единый механизм работы программы независимо от используемой библиотеки UI.

					МИВУ.09.03.02 015 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		15

3 Разработка приложения

3.1 Разработка системы на Java

Основой архитектуры программы стало использование объектно-ориентированного подхода и интерфейсов. Применение интерфейсов позволило разделить систему на независимые компоненты и обеспечить взаимодействие между ними через общий набор методов.

При проектировании было принято решение разделить программу на несколько основных подсистем:

- подсистема моделей данных (User, WorkoutProgram, Exercise, UserProgress);
- подсистема доступа к данным (DAO);
- подсистема работы с файлами;
- подсистема алгоритмов подбора тренировок (Strategy);
- подсистема пользовательских интерфейсов (View);
- подсистема экспорта отчётов.

Такое разделение позволило уменьшить связанность компонентов и упростить дальнейшее расширение проекта.

Для реализации алгоритмов подбора тренировок был создан интерфейс TrainingStrategy, определяющий общие методы для получения названия стратегии, рекомендаций, времени отдыха и расчёта интенсивности. Использование интерфейса позволило организовать единый механизм взаимодействия между программой и различными алгоритмами подбора. Интерфейс TrainingStrategy представлен на рисунке 5.

```

package com.workout.app.patterns.strategy;

import com.workout.app.model.User;

public interface TrainingStrategy {
    String getStrategyName();
    String getAdvice();
    int getRecommendedRestSeconds();
    String calculateIntensity(User user);
    String modifyExerciseReps(String originalReps);
}

```

Рисунок 5 – Интерфейс TrainingStrategy

Интерфейс реализуют классы MassGainStrategy, WeightLossStrategy, MaintenanceStrategy. Данные классы представлены на рисунках 6-8.

```

public class MassGainStrategy implements TrainingStrategy {
    @Override public String getStrategyName() { return "1. Классический (по цели)"; }
    @Override public String getAdvice() { return "Стандартная программа под твою цель из базы данных."; }
    @Override public int getRecommendedRestSeconds() { return 120; }

    @Override public String calculateIntensity(User user) {
        return String.format("Рабочий вес: %.1f кг (70% от текущего веса)", user.getWeight() * 0.7);
    }

    @Override public String modifyExerciseReps(String originalReps) { return originalReps + " [КЛАССИКА: стандартный объем]"; }
}

```

Рисунок 6 – Класс MassGainStrategy

```

public class WeightLossStrategy implements TrainingStrategy {
    @Override public String getStrategyName() { return "Классический (по цели)"; }
    @Override public String getAdvice() { return "Высокий темп, короткие паузы. Акцент на жиросжигание."; }
    @Override public int getRecommendedRestSeconds() { return 45; }
    @Override public String calculateIntensity(User user) {
        int hr = (int) ((220 - user.getAge()) * 0.7);
        return String.format("Целевой пульс: %d уд/мин (зона жиросжигания)", hr);
    }
    @Override public String modifyExerciseReps(String originalReps) {
        return originalReps.replace(target: "-", replacement: "+5") + " (высокий темп, больше повторов)";
    }
}

```

Рисунок 7 – Класс WeightLossStrategy

```

public class MaintenanceStrategy implements TrainingStrategy { no usages 1 polinaorl
    @Override public String getStrategyName() { return "Классический (по цели)"; } 2 usages 1 polinaorl
    @Override public String getAdvice() { return "Сбалансированная нагрузка. Фокус на технике и восстановлении."; }
    @Override public int getRecommendedRestSeconds() { return 75; } 2 usages 1 polinaorl
    @Override public String calculateIntensity(User user) { 1 usage 1 polinaorl
        return "Умеренная интенсивность. Поддерживай текущий тонус.";
    }
    @Override public String modifyExerciseReps(String originalReps) { 3 usages 1 polinaorl
        return originalReps + " (средний режим, техника в приоритете)";
    }
}

```

Рисунок 8 – Класс MaintenanceStrategy

Каждый из данных классов содержит собственную реализацию алгоритма подбора программы тренировок в зависимости от цели пользователя, однако все они работают через единый интерфейс. Благодаря этому основной код программы не зависит от конкретного алгоритма и может динамически выбирать нужную реализацию во время выполнения программы.

Особое внимание при проектировании уделялось независимости бизнес-логики от системы отображения данных. Для решения данной задачи был реализован интерфейс Observer. Через него система передаёт информацию об изменении параметров пользователя в модули визуализации.

Интерфейс Observer реализуют классы ProgressLoggerObserver, TrainingProgramObserver, AchievementObserver. Данные классы представлены на рисунках 9-12.

```

package com.workout.app.patterns.observer;

public interface Observer { 9 usages 3 implementation
    void update(String eventType, Object data);
}

```

Рисунок 9 – Класс Observer

```

public class ProgressLoggerObserver implements Observer { 3 usages  ▲ polinaorl
    private static final DateTimeFormatter formatter = 1 usage
        DateTimeFormatter.ofPattern("HH:mm:ss");

    @Override 1 usage  ▲ polinaorl
    public void update(String eventType, Object data) {
        String timestamp = LocalDateTime.now().format(formatter);

        switch (eventType) {
            case "WEIGHT_CHANGED" -> {
                System.out.println(" [" + timestamp + "] Вес изменён: " + data + " кг");
            }
            case "GOAL_CHANGED" -> {
                System.out.println(" [" + timestamp + "] Цель изменена: " + data);
            }
            case "LOCATION_CHANGED" -> {
                System.out.println(" [" + timestamp + "] Место изменено: " + data);
            }
            case "PROFILE_UPDATED" -> {
                System.out.println(" [" + timestamp + "] Профиль обновлён: " + data);
            }
            case "WORKOUT_COMPLETED" -> {
                System.out.println(" [" + timestamp + "] Тренировка завершена: " + data);
            }
            default -> {
                System.out.println(" [" + timestamp + "] " + eventType + " → " + data);
            }
        }
    }
}

```

Рисунок 10 – Класс ProgressLoggerObserver

```

public class TrainingProgramObserver implements Observer { 3 usages  ⚡ polinaorl
    private User user; 4 usages
    private Runnable onProgramUpdate; 3 usages

    public TrainingProgramObserver(User user, Runnable onProgramUpdate) { 3 usages  ⚡ polinaorl
        this.user = user;
        this.onProgramUpdate = onProgramUpdate;
    }

    @Override 1 usage  ⚡ polinaorl
    public void update(String eventType, Object data) {
        switch (eventType) {
            case "GOAL_CHANGED", "LOCATION_CHANGED" -> {
                System.out.println(" Пересчёт программы для пользователя: " + user.getEmail());
                // Обновляем данные пользователя
                if ("GOAL_CHANGED".equals(eventType)) {
                    user.setGoal((String) data);
                } else if ("LOCATION_CHANGED".equals(eventType)) {
                    user.setLocation((String) data);
                }
                // Запускаем пересчёт
                if (onProgramUpdate != null) {
                    onProgramUpdate.run();
                }
            }
        }
    }
}

```

Рисунок 11 – Класс TrainingProgramObserver.java

```

public class AchievementObserver implements Observer { 3 usages  ⚡ polinaorl
    private String userName; 3 usages

    public AchievementObserver(String userName) { this.userName = userName; }

    @Override 1 usage  ⚡ polinaorl
    public void update(String eventType, Object data) {
        switch (eventType) {
            case "WORKOUT_COMPLETED" -> {
                String message = switch ((int) (Math.random() * 3)) {
                    case 0 -> " Отличная работа, " + userName + "!";
                    case 1 -> " Ты становишься сильнее!";
                    default -> " Продолжай в том же духе!";
                };
                System.out.println(" Achievement: " + message);
                // В реальном приложении: показать красивое уведомление
                // JOptionPane.showMessageDialog(null, message, "Достижение!", JOptionPane.INFORMATION_MESSAGE);
            }
            case "PROFILE_UPDATED" -> {
                System.out.println(" Профиль " + userName + " обновлён");
            }
        }
    }
}

```

Рисунок 12 — Класс AchievementObserver.java

Во время работы приложения при изменении веса или других параметров система вызывает метод `update()` у всех подключённых наблюдателей. В этот момент визуализаторы получают актуальные данные и отображают изменения пользователю. Таким образом, бизнес-логика не зависит от конкретной библиотеки графического интерфейса и работает одинаково для всех вариантов отображения.

Для централизованного создания объектов были реализованы фабрики `GUIFactory`, `JavaFXGUIFactory`, `SwingGUIFactory`, `ConsoleGUIFactory`. Данные фабрики представлены на рисунках 13-16.

```
public interface GUIFactory {
    JFrame createLoginView();
    JFrame createRegistrationView();
    JFrame createProfileView(User user);
    JFrame createWorkoutProgramView(User user);
}
```

Рисунок 13 – Фабрика `GUIFactory`

```
public class JavaFXGUIFactory {
    private Stage primaryStage;

    public JavaFXGUIFactory(Stage primaryStage) {
        this.primaryStage = primaryStage;
    }

    public void showLogin() { new LoginFXMLView(primaryStage); }
    public void showRegistration() { new RegistrationFXMLView(primaryStage); }
    public void showProfile(User user) { new ProfileFXMLView(primaryStage, user); }
    public void showWorkoutProgram(User user) { new WorkoutProgramFXMLView(primaryStage, user); }
}
```

Рисунок 14 – Фабрика `JavaFXGUIFactory`

```

public class SwingGUIFactory implements GUIFactory { 2 usages  polinaorl

    @Override 1 usage  polinaorl
    public JFrame createLoginView() { return new LoginView(); }

    @Override 1 usage  polinaorl
    public JFrame createRegistrationView() { return new RegistrationView(); }

    @Override 1 usage  polinaorl
    public JFrame createProfileView(User user) { return new ProfileView(user); }

    @Override 1 usage  polinaorl
    public JFrame createWorkoutProgramView(User user) { return new WorkoutProgramView(user); }
}

```

Рисунок 15 – Фабрика SwingGUIFactory

```

public class ConsoleGUIFactory { 2 usages  polinaorl
    public void run() { 1 usage  polinaorl
        ConsoleLoginView login = new ConsoleLoginView();
        login.show();
    }
}

```

Рисунок 16 – Фабрика ConsoleGUIFactory

Использование фабрик позволило убрать из основного кода программы большое количество условных операторов и реализовать централизованное создание объектов. Основной модуль программы взаимодействует не с конкретными реализациями классов, а с интерфейсами, что значительно упрощает расширение системы.

Главным координирующим компонентом программы является класс MainJavaFX, представленный на рисунке 17.

```

public class MainJavaFX extends Application {  ± polinaorl
    @Override  ± polinaorl
    public void start(Stage primaryStage) {
        System.out.println("=== Система подбора тренировок (JavaFX) ===");
        try {
            System.out.println("Инициализация базы данных...");
            DatabaseInitializer.initialize();
            System.out.println("База данных успешно инициализирована!");
        } catch (Exception e) {
            System.err.println("Ошибка инициализации БД: " + e.getMessage());
        }

        JavaFXGUIImageFactory factory = new JavaFXGUIImageFactory(primaryStage);
        factory.showLogin();
        System.out.println("JavaFX приложение запущено!");
    }

    public static void main(String[] args) {  ± polinaorl
        launch(args);
    }
}

```

Рисунок 17 – Класс MainJavaFX

Для запуска консольной версии используется класс MainConsole, представленный на рисунке 18.

```

public class MainConsole {  ± polinaorl
    public static void main(String[] args) {  ± polinaorl
        System.out.println("\n=====");
        System.out.println(" СИСТЕМА ПОДБОРА ТРЕНИРОВОК");
        System.out.println(" Console Interface");
        System.out.println("===== \n");

        try {
            System.out.println("Инициализация базы данных...");
            DatabaseInitializer.initialize();
            System.out.println("База данных готова!\n");
        } catch (Exception e) {
            System.err.println("Ошибка: " + e.getMessage());
        }

        new ConsoleGUIImageFactory().run();
    }
}

```

Рисунок 18 — Класс MainConsole

Для запуска Swing-версии используется класс Main, представленный на рисунке 19.

```
public class Main {  ⚡ polinaorl
    public static void main(String[] args) {  ⚡ polinaorl
        SwingUtilities.invokeLater(() -> {
            System.out.println("=== Система подбора тренировок ===");
            System.out.println("Запуск с использованием Factory Pattern...\n");

            // Инициализация БД
            try {
                System.out.println("Инициализация базы данных...");
                DatabaseInitializer.initialize();
                System.out.println("База данных готова!\n");
            } catch (Exception e) {
                System.err.println("Ошибка инициализации БД: " + e.getMessage());
            }

            // Создаём приложение с Swing-фабрикой
            Application app = new Application(new SwingGUIFactory());

            // Запускаем окно входа
            app.showLogin();
            System.out.println("Приложение запущено!");
        });
    }
}
```

Рисунок 19 — Класс Main

Классы MainJavaFX, MainConsole и Main отвечают за инициализацию базы данных, создание фабрики интерфейсов и запуск приложения в соответствующем режиме. Именно эти классы связывают между собой загрузку профиля пользователя, выбор алгоритма подбора программы, визуализацию процесса и экспорт отчётов.

3.2 Разработка системы хранения данных

Следующим этапом стало проектирование подсистемы хранения и обработки данных. Поскольку программа должна поддерживать работу с базой данных Firebird и экспорт в различные форматы, было принято решение реализовать отдельную систему доступа к данным. Для этого были созданы классы UserDao, WorkoutProgramDao, UserProgressDao, ExerciseDao. Данные классы представлены на рисунках 20-23.

```
public Optional<User> findByEmail(String email) throws SQLException {
    String sql = "SELECT * FROM users WHERE email = ?";

    try (PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, email);

        try (ResultSet resultSet = statement.executeQuery()) {
            if (resultSet.next()) {
                return Optional.of(extractUser(resultSet));
            } else {
                return Optional.empty();
            }
        }
    }
}
```

Рисунок 20 – Класс UserDao

```

public Optional<WorkoutProgram> findByGoalAndLocation(String goal, String location) { 1 usage
    try {

        String sql = "SELECT FIRST 1 * FROM workout_programs WHERE goal = ? AND location = ?";
        var stmt = conn.prepareStatement(sql);
        stmt.setString( parameterIndex: 1, goal);
        stmt.setString( parameterIndex: 2, location);
        var rs = stmt.executeQuery();

        if (rs.next()) {
            return Optional.of(mapRow(rs));
        }
        return Optional.empty();
    } catch (Exception e) {
        e.printStackTrace();
        return Optional.empty();
    }
}

```

Рисунок 21 – Класс WorkoutProgramDAO

```

public void saveWeight(int userId, double weight, String note) throws SQLException { 30
    String sql = "INSERT INTO progress_logs (user_id, weight, notes) VALUES (?, ?, ?)";
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt( parameterIndex: 1, userId);
        stmt.setDouble( parameterIndex: 2, weight);
        stmt.setString( parameterIndex: 3, note);
        stmt.executeUpdate();
    }
}

```

Рисунок 22 – Класс UserProgressDAO

```

public List<Exercise> findAll() throws SQLException { 30 polinao
    String sql = "SELECT * FROM exercises ORDER BY name";
    List<Exercise> exercises = new ArrayList<>();

    try (Statement statement = connection.createStatement();
        ResultSet resultSet = statement.executeQuery(sql)) {

        while (resultSet.next()) {
            exercises.add(extractExercise(resultSet));
        }
    }

    return exercises;
}

```

Рисунок 23 — Класс ExerciseDAO

Класс UserDAO отвечает за операции с пользователями (регистрация, авторизация, обновление профиля), класс WorkoutProgramDAO — за работу с тренировочными программами, класс UserProgressDAO — за ведение журнала прогресса, а ExerciseDAO — за управление базой упражнений.

Для поддержки различных форматов экспорта в проекте реализованы методы saveProgressToExcel(), saveProgressToPdf(), saveProgressToDoc() внутри классов ConsoleProfileView и ProfileFXMLView.

```
private void saveProgressToExcel() throws Exception { 1 usage  ▲ polinaorl
    try (Workbook wb = new XSSFWorkbook()) {
        Sheet sh = wb.createSheet(s: "Прогресс тренировок");

        org.apache.poi.ss.usermodel.Font titleFont = wb.createFont();
        titleFont.setBold(true);
        titleFont.setFontHeightInPoints((short) 14);

        org.apache.poi.ss.usermodel.Font headerFont = wb.createFont();
        headerFont.setBold(true);
        headerFont.setFontHeightInPoints((short) 12);

        CellStyle titleStyle = wb.createCellStyle();
        titleStyle.setFont(titleFont);
        titleStyle.setFillForegroundColor(IndexedColors.DARK_BLUE.getIndex());
        titleStyle.setFillPattern(FillPatternType.SOLID_FOREGROUND);
        titleStyle.setAlignment(HorizontalAlignment.CENTER);
    }
}
```

Рисунок 24 — Метод экспорта в Excel

```
private void saveProgressToPdf() throws Exception { 1 usage  ▲ polinaorl
    String timestamp = new java.text.SimpleDateFormat(pattern: "yyyyMMdd_HH:mm:ss").format(new java.util.Date());
    String filename = "progress_" + user.getId() + "_" + timestamp + ".pdf";

    Document document = new Document();
    PdfWriter.getInstance(document, new FileOutputStream(filename));
    document.open();

    com.itextpdf.text.Font titleFont = new com.itextpdf.text.Font(
        com.itextpdf.text.Font.FontFamily.HELVETICA, size: 16, com.itextpdf.text.Font.BOLD);
    com.itextpdf.text.Font normalFont = new com.itextpdf.text.Font(
        com.itextpdf.text.Font.FontFamily.HELVETICA, size: 11, com.itextpdf.text.Font.NORMAL);
    com.itextpdf.text.Font boldFont = new com.itextpdf.text.Font(
        com.itextpdf.text.Font.FontFamily.HELVETICA, size: 12, com.itextpdf.text.Font.BOLD);
}
```

Рисунок 25 — Метод экспорта в PDF

```
private void saveProgressToDoc() throws Exception { 1 usage  ± polinaorl
    StringBuilder sb = new StringBuilder("<html><body style='font-family:Arial; padding:20px;'>");
    sb.append("<h2 style='color:#2c3e50;' История нпрорепсса: ").append(user.getEmail()).append("</h2>");
    sb.append("<p><b>Параметры:</b> ").append(user.getHeight()).append(" см | ")
        .append(user.getWeight()).append(" кг | ").append(user.getAge()).append(" лет</p>");
    sb.append("<h3> История веса:</h3>");
    sb.append("<table border='1' cellspacing='0' cellpadding='5'>");
    sb.append("<tr style='background:#e8f4f8;'><th>Дата</th><th>Вес (кг)</th><th>Изменение</th><th>Примечание</th></tr>");

    double prevWeight = user.getWeight();
    sb.append("<tr><td>При регистрации</td><td>").append(prevWeight).append("</td><td>0</td><td>Начальная точка</td></tr>");
```

Рисунок 26 — Метод экспорта в DOC

Работа подсистемы хранения данных организована следующим образом: при запуске программы через DatabaseManager устанавливается соединение с СУБД Firebird. DAO-классы считывают информацию и преобразуют её в объекты моделей (User, WorkoutProgram), находящиеся в оперативной памяти. После выполнения подбора программы или записи прогресса данные передаются соответствующему DAO-классу и сохраняются в базу. При необходимости экспорта отчёта пользователь выбирает формат, и соответствующий метод генерирует документ.

```
public class DatabaseManager {  ± polinaorl

    // Настройки подключения
    private static final String DB_HOST = "localhost"; 1 usage
    private static final String DB_PORT = "3050"; 1 usage
    private static final String DB_PATH = "D:\\workout.fdb"; 1 usage
    private static final String DB_USER = "SYSDBA"; 1 usage
    private static final String DB_PASSWORD = "masterkey"; 1 usage

    // Ссылка на базу данных
    private static final String URL = "jdbc:firebirdsql:" + DB_HOST + "/" + DB_PORT + ":" + DB_PATH;

    // Метод для получения подключения к базе данных

    public static Connection getConnection() throws SQLException {  ± polinaorl
        return DriverManager.getConnection(URL, DB_USER, DB_PASSWORD);
    }
}
```

Рисунок 27 — Класс DatabaseManager

Использование классов DAO позволило полностью отделить работу с данными от логики подбора программ и визуализации. Благодаря этому

добавление нового формата экспорта или смена СУБД требует только создания нового класса, реализующего соответствующую логику.

3.3 Разработка графического интерфейса

После завершения проектирования основной логики программы была разработана система визуализации и графический интерфейс приложения. Основной задачей данного этапа являлось создание удобного интерфейса для управления тренировочным процессом и обеспечение наглядного отображения прогресса.

В проекте реализованы три варианта интерфейса:

- консольная интерфейс;
- интерфейс на основе Swing;
- интерфейс на основе JavaFX.

Консольный интерфейс реализован в виде текстового меню в терминале. Во время работы программы пользователь видит текстовое представление профиля, список доступных упражнений и подобранную программу тренировок. Для повышения наглядности отображения была реализована цветовая подсветка важных элементов средствами ANSI-цветов терминала. Данный вариант интерфейса отличается минимальными требованиями к ресурсам системы и используется для быстрого доступа к функционалу программы.

Графический интерфейс Swing реализован в виде отдельного окна с использованием стандартных компонентов JFrame, JPanel, JButton, JLabel, JTextField, JComboBox, JTable. Интерфейс отображает параметры пользователя в виде полей ввода, таблицу с упражнениями и кнопки управления. Во время работы программы элементы интерфейса обновляются в режиме реального времени.

Наиболее функциональной частью проекта является интерфейс на основе JavaFX. Интерфейс JavaFX включает:

- экран авторизации и регистрации;

- главный экран с параметрами профиля;
- экран подобранной программы тренировок;
- экран журнала прогресса;
- кнопки экспорта отчётов;
- элементы управления.

На рисунках 28-30 представлены окна JavaFX-приложения. Интерфейс включает элементы управления профилем пользователя, выбором цели тренировок, отображением подобранной программы и кнопками экспорта отчётов. Центральная часть окна используется для отображения тренировочной программы в виде таблицы с упражнениями.

```
private void initUI() { 1 usage 1 polinaorl
    stage.setTitle("Профиль пользователя");

    VBox root = new VBox(20);
    root.setPadding(new Insets(30));
    root.setStyle("-fx-background-color: #f5f8fa;");

    Text title = new Text("Ваш профиль");
    title.setFont(javafx.scene.text.Font.font("Segoe UI", 24));
    title.setStyle("-fx-fill: #2c3e50; -fx-font-weight: bold;");

    VBox infoCard = new VBox(15);
    infoCard.setStyle("-fx-background-color: white; -fx-border-color: #c8d2dc; -fx-border-radius: 5; -fx-background-radius: 5;");
    infoCard.setPadding(new Insets(20));
    infoCard.setMaxWidth(500);
}
```

Рисунок 28 — Создание интерфейса профиля

```
private void addWorkoutSection(WorkoutProgram program, ProgramExerciseDAO dao) { 1 usage 1 polinaorl
    VBox card = new VBox(10);
    card.setStyle("-fx-background-color: white; -fx-border-color: #c8d2dc; -fx-border-radius: 5; -fx-background-radius: 5;");
    card.setPadding(new Insets(15));
    card.setMaxWidth(800);

    Text title = new Text(program.getName());
    title.setFont(Font.font("Segoe UI", 16));
    title.setStyle("-fx-font-weight: bold; -fx-fill: #2c3e50;");

    HBox infoBox = new HBox(20);
    infoBox.getChildren().addAll(
        new Text("Цель: " + program.getGoalText()),
        new Text("Место: " + program.getLocationText())
    );
    infoBox.setStyle("-fx-text-fill: #3c3c3c;");
}
```

Рисунок 29 — Добавление программы тренировок

```

private void showProgressSave() {
    ChoiceDialog<String> dialog = new ChoiceDialog<>("Excel", "Excel", "PDF", "DOC");
    dialog.setTitle("Сохранение прогресса");
    dialog.setHeaderText("Выберите формат сохранения:");
    dialog.setContentText("Формат:");

    Optional<String> result = dialog.showAndWait();
    result.ifPresent(method -> {
        try {
            switch (method) {
                case "Excel" -> saveProgressToExcel();
                case "PDF" -> saveProgressToPdf();
                case "DOC" -> saveProgressToDoc();
            }
        } catch (Exception e) {
            showAlert("Ошибка", e.getMessage());
        }
    });
}
}

```

Рисунок 30 — Метод экспорта

Для реализации графического интерфейса использовались компоненты JavaFX:

- Stage — главное окно приложения JavaFX;
- Scene — содержимое окна, в котором размещаются все элементы интерфейса;
- VBox, HBox — контейнеры для вертикального и горизонтального расположения элементов интерфейса;
- BorderPane, GridPane — контейнеры для компоновки элементов в виде сетки и границ;
- Button — кнопка для выполнения действий пользователя;
- TextField, PasswordField — поля ввода текстовых данных;
- Label — текстовая подпись для отображения названий и пояснений;
- ComboBox — выпадающий список для выбора цели тренировок и других параметров;
- Text — текстовый элемент для заголовков и описаний;
- TableView — таблица для отображения данных упражнений и

прогресса;

– Alert, TextInputDialog, ChoiceDialog — диалоговые окна для взаимодействия с пользователем.

Отображение данных реализовано с использованием JavaFX-компонентов, которые динамически обновляются при изменении состояния программы. Визуализаторы получают данные через интерфейс Observer и выполняют обновление графических элементов.

При проектировании графического интерфейса особое внимание уделялось независимости визуализации от логики подбора программ. Интерфейсы только отображают текущее состояние данных и не влияют на выполнение алгоритмов подбора. Благодаря этому программа обеспечивает одинаковую работу бизнес-логики независимо от используемой системы визуализации.

В результате разработки была создана модульная архитектура программы с чётким разделением ответственности между компонентами системы. Использование интерфейсов и паттернов проектирования позволило обеспечить гибкость архитектуры, независимость модулей и возможность дальнейшего расширения функциональности приложения.

					МИВУ.09.03.02 015 ПЗ	Лист
						32
Изм.	Лист	№ докум.	Подпись	Дата		

4 Тестирование

Одним из важнейших этапов создания программного продукта является его тестирование и отладка. Тестирование позволяет выявить скрытые и явные недостатки программы, либо убедиться в ее пригодности для применения.

Целью тестирования является проверка работоспособности программы, то есть правильности выполнения всех функций, описанных выше.

Таблица 1 – Тестирование

№	Действие	Результат
1.	Запуск JavaFX-версии приложения	Успех, отображение окна авторизации
2.	Попытка входа с неверным email/паролем	Вывод сообщения «Неверный email или пароль!»
3.	Регистрация нового пользователя	Успех, сохранение данных в БД
4.	Просмотр профиля пользователя	Успех, отображение параметров и ИМТ
5.	Запись нового веса в журнал прогресса	Успех, сохранение в БД и обновление интерфейса
6.	Просмотр программы тренировок	Успех, отображение упражнений с подходами
7.	Экспорт прогресса в Excel	Успех, создание файла
8.	Панель администратора	Успех, отображение таблицы пользователей
9.	Запуск консольной версии приложения	Успех, отображение текстового меню

Вывод сообщений об ошибках и другие сообщения представлены на рисунках 31-39.

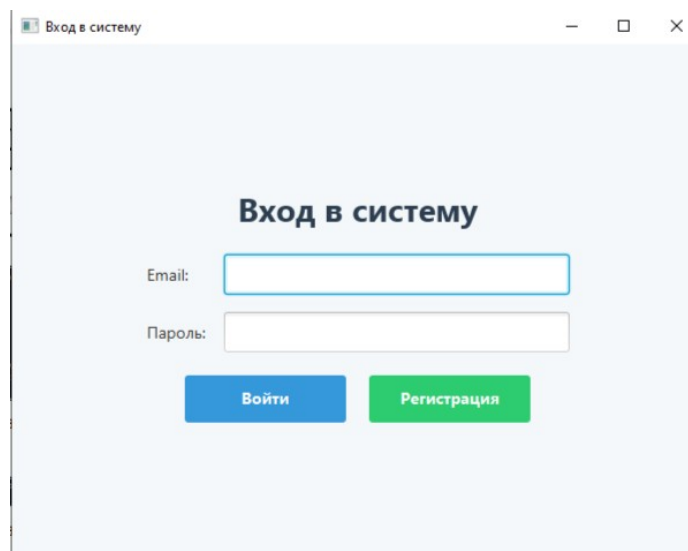


Рисунок 31 – Вывод результата теста №1

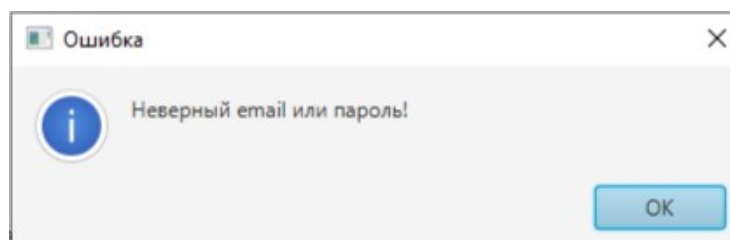


Рисунок 32 – Вывод результата теста №2

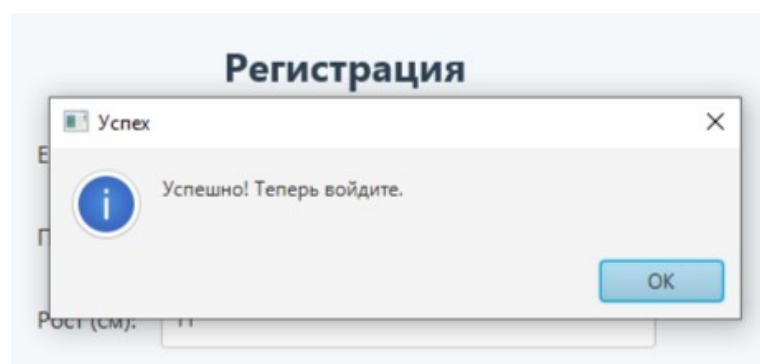


Рисунок 33 – Вывод результата теста №3

Ваш профиль

Email:
1

Цель:
Набор мышечной массы

Место:
Спортивный зал

Параметры:
11 см | 11 кг | 11 лет

ИМТ: 909,1 (Ожирение)

Рисунок 34 – Вывод результата теста №4

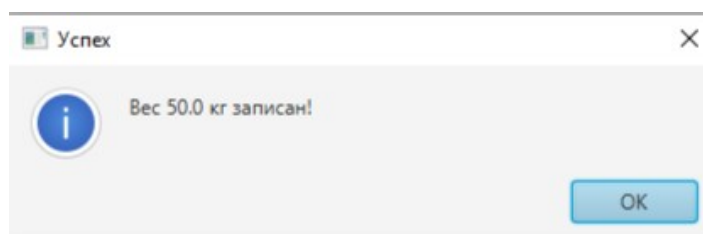


Рисунок 35 – Вывод результата теста №5

Движок расчёта

День 1: Грудь + Трицепс

Цель: Набор массы Место: Зал

- Приседания со штангой** 4 x 8-10 [КЛАССИКА: стандартный объём] | 6 кг | Ноги
- Подъём штанги на бицепс** 3 x 10-12 [КЛАССИКА: стандартный объём] | 2 кг | Руки
- Разведение гантелей в стороны** 3 x 10-12 [КЛАССИКА: стандартный объём] | 4 кг | Плечи
- Выпады с гантелями** 3 x 12-15 [КЛАССИКА: стандартный объём] | 4 кг | Ноги

День 2: Спина + Бицепс

Цель: Набор массы Место: Зал

- Отжимания от пола** 4 x 8-10 [КЛАССИКА: стандартный объём] | 4 кг | Грудь

Рисунок 36 – Вывод результата теста №6

24	День 3: Ноги + Плечи						
25	1	Становая тяга	Спина	4	8-10	4	Штанга
26	2	Тяга гантели в наклоне	Спина	3	12-15	4	Гантели
27	3	Подтягивания	Спина	3	10-12	4	Собственный вес
28	4	Планка	Пресс	3	12-15	4	Собственный вес
29							
30							
31	ИСТОРИЯ ВЕСА						
32	Дата	Вес (кг)	Изменение	Примечание			
33	При регистрации	11	0	Начальная точка			
34	04.06.2026 18:19	50	+39,0	Записано из JavaFX			

Рисунок 37 – Вывод результата теста №7

Панель администратора							
ПАНЕЛЬ АДМИНИСТРАТОРА							
Пользователи Упражнения							
ID	Email	Цель	Место	Рост	Вес	Возраст	
1	1	Набор мышечной массы	Спортивный зал	11.0	11.0	11	
2	ggjgj@yandex.ru	Набор мышечной массы	Спортивный зал	212.0	145.0	20	

Рисунок 38 – Вывод результата теста №7

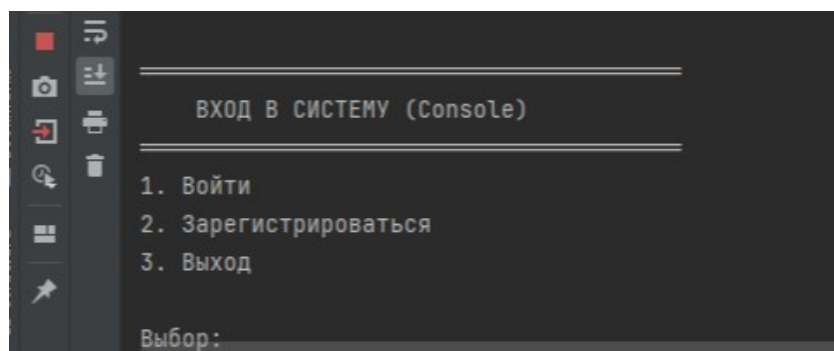


Рисунок 39 – Вывод результата теста №7

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы разработана программная система подбора тренировочных программ и ведения дневника тренировок на Java. Целью проекта стало создание расширяемого приложения для обработки персональных данных, динамического подбора нагрузок по целям и условиям, а также удобного контроля прогресса.

В процессе разработки была спроектирована модульная архитектура, включающая подсистемы моделей, доступа к БД, алгоритмов подбора, интерфейсов и экспорта. Такой подход позволил разделить ответственность между компонентами и обеспечил их независимость.

В рамках работы были реализованы три стратегии подбора программ: MassGainStrategy, WeightLossStrategy и MaintenanceStrategy. Для взаимодействия компонентов использовались интерфейсы TrainingStrategy, Observer, UserDAO, WorkoutProgramDAO. Программа подбирается по параметрам пользователя, а интерфейс лишь отображает результат, не влияя на алгоритмы.

Для взаимодействия с пользователем были реализованы три варианта интерфейса: консольный, на базе Swing и JavaFX. Также добавлен экспорт отчётов в Excel, PDF и Word с использованием библиотек Apache POI и iText.

При разработке были использованы паттерны проектирования Factory, Observer и Strategy, что обеспечило гибкость архитектуры и возможность дальнейшего расширения программы. Дополнительно были настроены Maven, локальная СУБД Firebird для хранения данных и система паттернов для обеспечения слабосвязанной архитектуры.

В результате выполнения курсовой работы была создана полнофункциональная система подбора тренировочных программ, соответствующая поставленным требованиям и демонстрирующая применение принципов объектно-ориентированного программирования, паттернов проектирования и современных средств разработки программного обеспечения на платформе Java.

					МИВУ.09.03.02 015 ПЗ	Лист
						37
Изм.	Лист	№ докум.	Подпись	Дата		

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ермаков, А. В. Разработка приложений на языке JAVA : учебное пособие / А. В. Ермаков, М. С. Королёв, А. К. Кузьмин. — Саратов : Саратовский государственный технический университет имени Ю.А. Гагарина, ЭБС АСВ, 2024. — 128 с. — ISBN 978-5-7433-3635-7. — Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. — URL: <https://www.iprbookshop.ru/150088.html>
2. Java. Библиотека профессионала. Том 1. Основы [Электронный ресурс]: / К. С. Хорстманн, Г. Корнелл. — Электрон. текстовые данные. — М.: Вильямс, 2019. — 832 с. — Режим доступа: <https://www.litres.ru/key-horstmann/java-biblioteka-professionala-tom-1-osnovy/>
3. Oracle. Java Swing Tutorial [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/javase/tutorial/uiswing/?utm_source=chatgpt.com
4. Oracle. JavaFX Documentation [Электронный ресурс]. — Режим доступа: https://openjfx.io/?utm_source=chatgpt.com

